

The RDF-3X engine for scalable management of RDF data

Authored by: Thomas Neumann and
Gerhard Weikum

Presented by: Kyle Robinson

Overview

- Motivation
- Challenges
- Previous Work
- RDF-3X
- Storage
- Querying
- Query Optimizer
- Cost estimation
- Updates
- Results
- Conclusion

Motivation

- Create a way for machines to understand the semantics of information
- The goal of RDF is to provide a way to describe resources
- RDF model is very important to the semantic web
- Previous RDF databases performed poorly
- Managing large scale RDF data is necessary and challenging

Challenges

- Absence of a global schema causes issue for physical database design
- Optimizations require statistics gathering without schema
- Lots of joins but attributes are less predictable than in relational setting
- Does not benefit from the XML research done for trees

Previous Work

- Triples are stored in a single triples table
- Map triples onto relational tables (Sesame, SW-Store)
- All triples with the same predicate stored in same property table
- An extension where triples are clustered by predicate names

RDF-3X

- Creates giant triples table
- Indexes all of it using 6 permutations of S, P, O
- Indexes over all two- and one-dimensional projections
- Easy compression, smaller than original data
- Allows for batch updates

Storage

- Triples stored in a B+ tree
- Literals all replaced by IDs in a mapping dictionary
- IDs act as compression and simplification for the query processor
- Compress indexes by storing the change between triples
- If there is only a small change in value 3 then the delta can be stored in the header

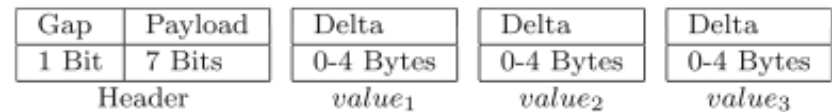


Fig. 1 Structure of a compressed triple

Querying

- Queries translated into triple patterns
- Single triple pattern solved by range scan
- Multiple triple patterns need multiple joins
- Construct an execution plan from query graph
 - Create index scan for each triple pattern
 - Join each edge in the graph
 - Add cross-products to obtain single join tree
 - Selection for FILTER
 - Eliminate duplicates, DISTINCT
 - Convert IDs to literals
- First steps taken care of by query optimizer

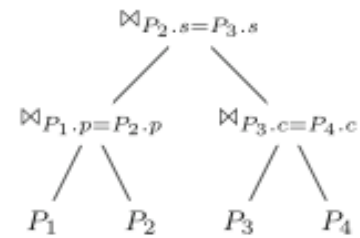
Example of Querying

```
select ?n Where {
  ?p <hasName> ?n.
  ?s <performedBy> ?p.
  ?s <composedBy> ?c.
  ?c <hasName> "Bob Dylan".
}
```

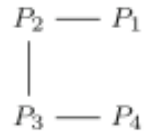
(a)

$P_1 = (?p, \text{hasName}, ?n)$
 $P_2 = (?s, \text{performedBy}, ?p)$
 $P_3 = (?s, \text{composedBy}, ?c)$
 $P_4 = (?c, \text{hasName}, \text{Bob Dylan})$

(b)



(c)



(d)

Fig. 4 SPARQL translation example. **a** SPARQL, **b** Triples form, **c** Possible join tree, **d** Query graph

Query Optimizer

- Uses a bottom-up dynamic programming framework
- Solve subgraphs of query graph
- Projects away unused variables and generates plans for **all** indexes
- Prunes plans by estimating execution costs
- Optimal solutions to smaller problems are joined with each other

Execution Plan Examples

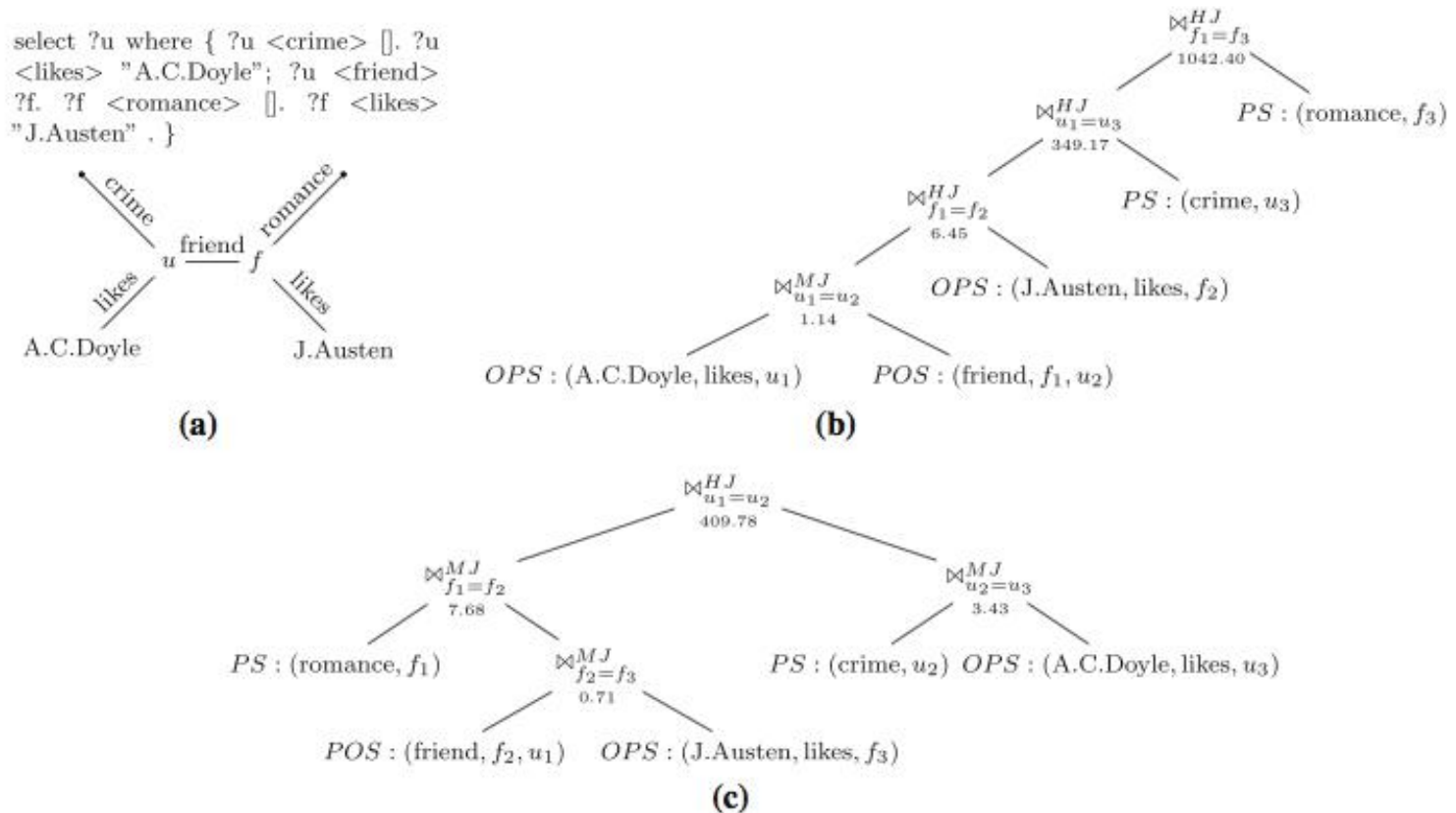


Fig. 5 Execution plans with estimated CPU costs for LibraryThing query B1. **a** Original query, **b** Plan constructed by minimum-selectivity-ordering, **c** Fully optimized plan

Cost Estimation

- Use histograms to generate statistics on groups of triples
- General but assumes predicates are independent
- Augment histograms with precomputed frequent join paths
- Rank paths by number of distinct nodes
- Can generate 1000 paths in a few minutes and takes a 16KB page

start (s,p,o)		end (s,p,o)	
number of triples			
number of distinct 2-prefixes			
number of distinct 1-prefixes			
join partners on subject			
	s=s	s=p	s=o
join partners on predicate			
	p=s	p=p	p=o
join partners on object			
	o=s	o=p	o=o

Fig. 7 Structure of a histogram bucket

Estimation Error

Table 3 Average relative estimation errors for query patterns from Sect. 8

	Barton	Yago	Librarything
Single-dimensional histograms	0.44	2.18	15.74
RDF selectivity histograms	0.73	0.86	0.88
+ Frequent paths	0.57	0.78	0.54

Updates

- Updates collected in private workspaces
- Differential indexing for single and batch updates
- Merge into main index when size meets threshold
- Indexed separate and lazily from the main index
- Differential indexes invalidated upon new update
- Deletes are just updates with a delete flag

Results

Table 5 Query run-times in seconds for the Barton dataset

	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Geom. mean
Cold caches								
RDF-3X	0.14	3.10	31.49	11.57	18.82	2.75	32.61	5.9
MonetDB	5.66	11.70	54.66	34.77	80.96	14.14	61.52	26.4
PostgreSQL	28.32	181.00	291.04	224.61	199.07	207.72	271.20	167.8
Warm caches								
RDF-3X	0.001	1.17	2.22	1.58	0.49	1.20	1.26	0.4
MonetDB	0.65	1.41	3.63	9.59	77.53	1.86	2.48	3.8
PostgreSQL	8.15	174.41	286.76	26.80	8.77	206.46	231.79	64.3

Table 6 Query run-times in seconds for the Yago dataset

	A1	A2	A3	B1	B2	B3	C1	C2	Geom. mean
Cold caches									
RDF-3X	0.29	0.28	1.20	0.28	0.99	0.33	2.23	4.23	0.73
MonetDB	43.55	44.13	55.49	62.94	182.39	72.22	101.66	157.11	78.29
PostgreSQL	1.62	6.31	5.46	3.04	117.51	4.71	29.84	59.64	10.66
Warm caches									
RDF-3X	0.02	0.02	0.02	0.01	0.05	0.01	0.61	1.44	0.04
MonetDB	36.92	32.96	34.72	49.95	64.84	52.22	84.41	131.35	54.62
PostgreSQL	0.08	0.43	0.20	0.11	7.33	0.12	0.31	50.37	0.56

Table 7 Query run-times in seconds for the LibraryThing dataset

	A1	A2	A3	B1	B2	B3	C1	C2	Geom. mean
Cold caches									
RDF-3X	0.28	1.01	21.85	0.14	0.34	4.17	0.28	1.21	0.89
MonetDB	2.14	1.41	1220.09	1.63	2.20	*	1.66	>15 min/*	>8.16
PostgreSQL	20.78	1.43	715.64	0.88	2.13	>8h	5108.01	1031.63	>93.91
Warm caches									
RDF-3X	0.05	0.15	0.95	0.01	0.12	1.61	0.03	0.26	0.13
MonetDB	0.82	0.77	1171.82	0.56	0.63	*	0.59	>15 min/*	>4.39
PostgreSQL	12.31	0.05	611.41	0.02	0.66	>8h	5082.34	1013.01	>30.43

* System crashed, see description

Conclusion

- RDF-3X allows for executing SPARQL queries over large amounts of RDF triples
- Outperforms the best previous systems by a large margin
- Uses exhaustive sextuple indexing
- Query optimizer takes advantage of fast merge joins
- Allows for batch updates and deletions